



HTML Form และการส่งข้อมูลผ่าน GET/POST



Asst.Prof. Monlica Wattana, Ph.D

Department of Computer Science, Khon Kaen University

CP423324 Database and Web Security





Outline



- พื้นฐาน HTML Form
- ประเภทข้อมูลเข้า (Input Types)
- วิธีการส่งข้อมูลแบบ GET และ POST
- การตรวจสอบและจัดการข้อมูลจากผู้ใช้ใน Web Form





การทำงานของ HTML Form



การทำงานของ Form ตั้งแต่ผู้ใช้กรอกข้อมูลจนถึง Server

1. ผู้ใช้กรอกข้อมูลใน Form
2. Browser รวบรวมค่าจาก input ที่มี name
3. Browser สร้าง HTTP Request
4. ข้อมูลถูกส่งด้วย GET หรือ POST
5. Server รับข้อมูลและตรวจสอบ
6. Server ประมวลผลหรือบันทึกข้อมูล





HTML Form เบื้องต้น



HTML Form

- `<form>` เป็นคอนเทนเนอร์สำหรับองค์ประกอบอินพุตประเภทต่างๆ เช่น ช่องข้อความ ช่องทำเครื่องหมาย ปุ่มตัวเลือก ปุ่มส่งข้อมูล เป็นต้น
- โครงสร้างพื้นฐานของ Form

```
<form action="..." method="..."> </form >
```

action: ระบุ URL ของไฟล์บน Server ที่จะทำหน้าที่ประมวลผลข้อมูลที่ส่งไป (เช่น process.php หรือ /api/save)

method: ระบุวิธีการส่งข้อมูล (HTTP Method) ซึ่งมาตรฐานที่ใช้กันบ่อยที่สุดคือ GET และ POST





HTML Form เบื้องต้น



วิธีการส่งข้อมูล(method)

วิธีการส่งข้อมูลแบบ GET

- ข้อมูลจะถูกส่งไปต่อท้าย URL (Query String)
- มองเห็นข้อมูลได้ชัดเจนบนแถบ Address Bar
- ทำ Bookmark และเก็บ Cache ได้

วิธีการส่งข้อมูลแบบ Post

- ส่งข้อมูลภายในส่วน Body ของคำขอ HTTP
- ไม่แสดงข้อมูลบน URL เหมาะกับข้อมูลสำคัญ
- ส่งข้อมูลขนาดมากกว่า GET
- ใช้สำหรับอัปโหลดไฟล์หรือเขียนข้อมูลลง Database





HTML Form เบื้องต้น

Element ของ HTML สามารถประกอบอยู่ใน <form> อย่างน้อยหนึ่งรายการต่อไปนี้

Form Element	การใช้งาน	ตัวอย่างการใช้งาน
<select>	สร้างรายการตัวเลือกแบบ Drop-down List	เลือกหมวดหมู่ จังหวัด หรือประเภทข้อมูล
<option>	กำหนดตัวเลือกแต่ละรายการภายใน <select>	HTML, CSS, Web Security
<textarea>	ใช้สร้างช่องกรอกข้อความหลายบรรทัด	กล่องข้อความ Message หรือ Comment
<button>	ใช้สร้างปุ่มกด	Submit, Reset, Cancel หรือปุ่มที่ใช้กับ JavaScript
<fieldset>	ใช้จัดกลุ่มข้อมูลในฟอร์ม	กลุ่มข้อมูลส่วนตัว กลุ่มข้อมูลติดต่อ
<legend>	ใช้ตั้งชื่อหัวข้อให้กับกลุ่มข้อมูลใน <fieldset>	Personal Information, Contact Information



Fiat ▾

Volvo

Saab

Fiat

Audi

At w3schools.com you will learn how to make a website. They offer free tutorials in all web development technologies.

Click Me!

Personalia:

First name:

Birthdate:



HTML Form เบื้องต้น

Element ของ HTML สามารถประกอบอยู่ใน `<form>` อย่างน้อยหนึ่งรายการต่อไปนี้

Form Element	การใช้งาน	ตัวอย่างการใช้งาน
<code><datalist></code>	ใช้สร้างรายการตัวเลือกแนะนำให้กับ <code><input></code>	ช่องค้นหาที่มีคำแนะนำอัตโนมัติ
<code><output></code>	ใช้แสดงผลลัพธ์จากการคำนวณหรือการประมวลผล	แสดงผลรวมราคา หรือค่าที่เลือกจาก range
<code><option></code>	ใช้กำหนดตัวเลือกภายใน <code><select></code> หรือ <code><datalist></code>	HTML, CSS, Web Security
<code><optgroup></code>	ใช้จัดกลุ่มตัวเลือกภายใน <code><select></code>	กลุ่ม Front-end, Back-end, Security

Form with a dropdown menu and a Submit button. The dropdown menu is open, showing options: Edge, Firefox, and Chrome.

Form with a dropdown menu showing a list of cars. The dropdown menu is open, showing options: Swedish Cars (Volvo, Saab) and German Cars (Mercedes, Audi). Audi is selected.





HTML Form เบื้องต้น



- Element ของ HTML ที่สำคัญคือ `<label>` ใช้สำหรับกำหนดป้ายกำกับให้กับองค์ประกอบในแบบฟอร์มหลายอย่าง
- แอตทริบิวต์ `for` ของแท็ก `<label>` ควรเท่ากับแอตทริบิวต์ `id` ของแท็ก `<input>` เพื่อเชื่อมโยงเข้าด้วยกัน

```
<label for="fullname">Full Name:</label>
```





HTML Form เบื้องต้น



ตัวอย่างการใช้งาน <label>, <select> และ <option>

```
<label for="category">Category:</label>
<select id="category" name="category">
  <option value="html">HTML</option>
  <option value="css">CSS</option>
  <option value="web-security">Web Security</option>
</select>
```

Category:

HTML	▼
HTML	
CSS	
Web Security	





HTML Form เบื้องต้น



- `<input>` ใน HTML เป็นองค์ประกอบแบบฟอร์มที่ใช้บ่อยที่สุด และสามารถแสดงผลได้หลายวิธี ขึ้นอยู่กับแอตทริบิวต์ `type`

Input Type	การใช้งาน	ตัวอย่างการใช้งาน
<code><input type="button"></code>	สร้างปุ่มทั่วไป ต้องใช้ร่วมกับ JavaScript เพื่อให้ทำงาน	ปุ่มกดแสดงข้อความ
<code><input type="checkbox"></code>	สร้างตัวเลือกที่เลือกได้หลายข้อ	เลือกความสนใจหลายรายการ
<code><input type="color"></code>	เลือกค่าสีจาก Color Picker	เลือกสีพื้นหลัง
<code><input type="date"></code>	เลือกวันที่	วันเกิด, วันที่จอง
<code><input type="datetime-local"></code>	เลือกวันที่และเวลา	วันและเวลานัดหมาย





HTML Form เบื้องต้น



- แอตทริบิวต์ type ของ <input>

Input Type	การใช้งาน	ตัวอย่างการใช้งาน
<input type="email">	รับข้อมูลอีเมล	student@kku.ac.th
<input type="file">	อัปโหลดไฟล์	อัปโหลดรูปภาพหรือเอกสาร
<input type="hidden">	ซ่อนข้อมูลไม่ให้ผู้ใช้เห็น แต่ส่งไปพร้อม Form	ส่งค่า user_id หรือ token
<input type="image">	ใช้รูปภาพเป็นปุ่ม Submit	ปุ่มส่งข้อมูลแบบรูปภาพ
<input type="month">	เลือกเดือนและปี	เดือนที่ชำระเงิน
<input type="number">	รับตัวเลข	อายุ, จำนวนสินค้า
<input type="password">	รับรหัสผ่าน โดยซ่อนตัวอักษรขณะพิมพ์	Password



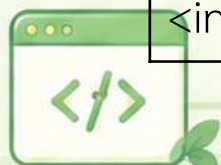


HTML Form เบื้องต้น



- แอตทริบิวต์ type ของ <input>

Input Type	การใช้งาน	ตัวอย่างการใช้งาน
<input type="range">	เลือกค่าจากแถบเลื่อน	ระดับความพึงพอใจ
<input type="reset">	ป้อนล้างค่าข้อมูลใน Form	Reset
<input type="search">	ช่องค้นหาข้อมูล	ค้นหาคำว่า login
<input type="submit">	ป้อนส่งข้อมูล Form	Submit, Register, Login
<input type="tel">	รับเบอร์โทรศัพท์	0899999999
<input type="text">	รับข้อความทั่วไป	ชื่อ, Username
<input type="time">	เลือกเวลา	เวลาเริ่มเรียน
<input type="url">	รับ URL หรือที่อยู่เว็บไซต์	https://www.kku.ac.th





HTML Form เบื้องต้น



แอตทริบิวต์ **name** ของ `<input>`

- **name** Attribute คือ ชื่อของข้อมูลที่ถูกส่งไปกับ Form
- ใน HTML Form ค่าที่จะถูกส่งไปยัง Server จะอ้างอิงจาก attribute name ตัวอย่าง

```
<input type="text" name="username">  
<input type="password" name="password">
```

เมื่อกด Submit แบบ GET อาจได้ URL เช่น

```
login.html?username=alice&password=123456
```

ถ้า input ไม่มี name ข้อมูลนั้นจะไม่ถูกส่งไปพร้อม Form





HTML Form เบื้องต้น



ความแตกต่างระหว่าง id, name, และ value

Attribute	หน้าที่	ตัวอย่าง
id	ใช้ระบุ element ในหน้าเว็บ และเชื่อมกับ <label>	id="email"
name	ใช้เป็นชื่อข้อมูลที่ถูกส่งไปกับ Form	name="email"
value	ใช้กำหนดค่าของข้อมูลที่ส่งไป	value="user"

ตัวอย่าง:

```
<input type="radio" id="user" name="role" value="user">  
<label for="user">User</label>
```





การตรวจสอบและจัดการข้อมูลจากผู้ใช้ใน Web Form



Untrusted Input: ข้อมูลจากผู้ใช้ไม่น่าเชื่อถือเสมอ

ข้อมูลจากผู้ใช้ หมายถึง ข้อมูลที่มาจาก Form, URL, Query String, Request Body, Cookie หรือ DevTools

ตัวอย่างข้อมูลจากผู้ใช้:

username=alice

password=123456

role=user

keyword=

email=student@kku.ac.th





การตรวจสอบและจัดการข้อมูลจากผู้ใช้ใน Web Form



การตรวจสอบข้อมูลของ Client-Side และ Server-Side

- **Client-Side Validation**

เป็นการตรวจสอบข้อมูลที่เกิดขึ้นบน Browser ของผู้ใช้ เช่น การใช้ HTML Attribute หรือ JavaScript เพื่อตรวจสอบก่อนส่งข้อมูลไปยัง Server

ข้อดี

- ช่วยแนะนำผู้ใช้ให้กรอกข้อมูลถูกต้อง
- ลดความผิดพลาดเบื้องต้น
- ทำให้ใช้งานสะดวกและรวดเร็ว
- ลด Request ที่ไม่จำเป็นไปยัง Server

ข้อจำกัด

- ผู้ใช้สามารถแก้ไข HTML ผ่าน DevTools ได้
- สามารถปิด JavaScript ได้
- สามารถส่ง Request เองโดยไม่ผ่านหน้าเว็บได้
- จึงไม่ควรใช้ Client-Side เป็นการป้องกันหลัก





การตรวจสอบและจัดการข้อมูลจากผู้ใช้ใน Web Form



การตรวจสอบข้อมูลของ Client-Side และ Server-Side

- **Server-Side Validation**

เป็นการตรวจสอบข้อมูลที่ฝั่ง Server หลังจากได้รับข้อมูลจาก Form หรือ Request แล้ว ก่อนนำไปใช้ บันทึกลงฐานข้อมูล หรือใช้ตัดสินใจสิทธิ์ของผู้ใช้

สิ่งที่ควรตรวจสอบฝั่ง Server

- ตรวจสอบว่าข้อมูลครบถ้วนหรือไม่
- ตรวจสอบรูปแบบ เช่น Email, เบอร์โทร, วันที่
- ตรวจสอบความยาวของข้อมูล
- ตรวจสอบชนิดข้อมูล เช่น ตัวเลข ข้อความ ไฟล์
- ตรวจสอบสิทธิ์ เช่น role, user_id, permission
- ป้องกันการโจมตี เช่น SQL Injection, XSS, File Upload Attack





การตรวจสอบและจัดการข้อมูลจากผู้ใช้ใน Web Form



ประเด็นสำคัญที่ต้องควบคุมความปลอดภัย

- ผู้ใช้สามารถกรอกข้อมูลผิดรูปแบบได้
- ผู้ใช้สามารถแก้ไข URL ได้
- ผู้ใช้สามารถแก้ไข Request ผ่าน DevTools หรือเครื่องมืออื่นได้
- ค่าที่ส่งมา เช่น role=admin ไม่ได้แปลว่าผู้ใช้เป็น Admin จริง
- ระบบไม่ควรเชื่อข้อมูลจากผู้ใช้ทันที

** ข้อมูลจากผู้ใช้ต้องไม่น่าเชื่อถือเสมอ จนกว่า Server จะตรวจสอบแล้ว





การตรวจสอบและจัดการข้อมูลจากผู้ใช้ใน Web Form



ตัวอย่าง: ผู้ใช้สามารถแก้ไขข้อมูลใน URL ได้

ตัวอย่าง ข้อมูลที่ส่งเดิม

..... ?username=alice&password=123456&role=user

ผู้ไม่หวังดีแก้ไข

.....?username=admin&password=admin123&role=admin





การตรวจสอบและจัดการข้อมูลจากผู้ใช้ใน Web Form



สิ่งสำคัญที่ต้องพิจารณา

- URL สามารถมองเห็นได้
- URL สามารถแก้ไขได้
- URL อาจถูกบันทึกใน Browser History
- ข้อมูลสำคัญไม่ควรส่งผ่าน GET
- สิทธิ์ของผู้ใช้ต้องตรวจสอบจากฝั่ง Server





การตรวจสอบและจัดการข้อมูลจากผู้ใช้ใน Web Form

- HTML ช่วยจัดรูปแบบ Input ได้ แต่ไม่ใช่ระบบความปลอดภัย

เครื่องมือช่วยควบคุมข้อมูลเบื้องต้น

คำสั่ง HTML	ตัวอย่าง	ทำงาน
type="email"	<code><input type="email"></code>	บังคับรูปแบบอีเมลเบื้องต้น
type="number"	<code><input type="number"></code>	รับตัวเลข
type="password"	<code><input type="password"></code>	ซ่อนตัวอักษรขณะพิมพ์
required	<code><input required></code>	บังคับให้กรอกข้อมูล
maxlength	<code><input maxlength="20"></code>	จำกัดจำนวนตัวอักษร
min, max	<code><input type="number" min="1" max="100"></code>	จำกัดช่วงตัวเลข
pattern	<code><input pattern="[0-9]{10}"></code>	กำหนดรูปแบบข้อมูลด้วย Regular Expression
placeholder	<code><input type="text" placeholder="Enter username"></code>	แสดงข้อความตัวอย่างหรือคำแนะนำในช่องกรอกข้อมูล





การตรวจสอบและจัดการข้อมูลจากผู้ใช้ใน Web Form



ตัวอย่าง

```
<input type="email" name="email" required>
```

```
<input type="tel" name="phone"  
  pattern="[0-9]{10}"  
  maxlength="10">
```

```
<input type="password" name="password"  
  minlength="8" required>
```

- HTML Validation ช่วยลดความผิดพลาดของผู้ใช้
- แต่ไม่สามารถป้องกันการแก้ไข Request ได้ทั้งหมด





การตรวจสอบและจัดการข้อมูลจากผู้ใช้ใน Web Form



ข้อจำกัดของการตรวจสอบด้วย HTML

แม้จะใช้ `required`, `type="email"`, `pattern` หรือ `maxlength` แล้ว ข้อมูลก็ยังไม่น่าเชื่อถือเสมอ เพราะผู้ใช้สามารถ

- ปิด validation ใน Browser ได้
- แก้ HTML ผ่าน DevTools ได้
- แก่ค่า `name`, `value`, `role` ได้
- ส่ง Request เองโดยไม่ผ่านหน้าเว็บได้
- ส่งค่าที่ไม่ตรงกับที่หน้าเว็บกำหนดได้





การตรวจสอบและจัดการข้อมูลจากผู้ใช้ใน Web Form



ข้อจำกัดของการตรวจสอบด้วย HTML

ตัวอย่าง

- หน้าเว็บอาจมี radio ให้เลือกเพียง:
`<input type="radio" name="role" value="user">`
`<input type="radio" name="role" value="admin">`
- แต่ผู้ใช้สามารถส่งค่าอื่นได้ เช่น
`role=superadmin`
- ดังนั้น HTML ช่วยจัดการข้อมูลเบื้องต้นได้ แต่ Server ต้องตรวจสอบซ้ำเสมอ





การตรวจสอบและจัดการข้อมูลจากผู้ใช้ใน Web Form



แนวทางจัดการ Input อย่างปลอดภัย

- Validate ตรวจสอบว่าข้อมูลถูกต้องตามรูปแบบ เช่น email, phone, password length
- Sanitize ทำความสะอาดข้อมูล เช่น ตัดช่องว่าง ลบ/แปลงอักขระที่ไม่ต้องการ
- Authorize ตรวจสอบสิทธิ์จากฝั่ง Server ไม่เชื่อค่า role จากผู้ใช้
- Use POST for sensitive data ข้อมูลสำคัญไม่ควรอยู่ใน URL
- Use HTTPS ป้องกันข้อมูลถูกดักระหว่างทาง
- Never trust user input ข้อมูลจากผู้ใช้ต้องถูกตรวจสอบก่อนใช้งานเสมอ





การตรวจสอบและจัดการข้อมูลจากผู้ใช้ใน Web Form



แนวทางปฏิบัติที่ดีในการสร้าง Form

- ใช้ `<label>` คู่กับ input ทุกช่อง
- กำหนด name ให้ input ที่ต้องการส่งข้อมูล
- ใช้ input type ให้เหมาะสม เช่น email, tel, password
- ใช้ required, maxlength, pattern เพื่อช่วยตรวจสอบเบื้องต้น
- ใช้ GET กับข้อมูลที่ไม่เป็นความลับ
- ใช้ POST กับข้อมูลที่ไม่ควรแสดงใน URL
- ไม่ส่ง password หรือ role ผ่าน URL
- ตรวจสอบข้อมูลซ้ำที่ Server ใช้ HTTPS กับข้อมูลสำคัญ
- ไม่เชื่อข้อมูลจาก Browser โดยตรง



ตัวอย่าง HTML Form

Student Register Form

Full Name:

Email:

Password:

Birthday:

Role:

☒ User ☐ Admin

Interested Topics:

☐ HTML ☐ CSS ☐ Web Security

Favorite Color:



Submit

Reset

input type	หน้าที่
text	กรอกข้อความทั่วไป เช่น ชื่อ
email	กรอกอีเมล
password	กรอกรหัสผ่าน โดยซ่อนตัวอักษร
date	เลือกวันที่
radio	เลือกได้ 1 ตัวเลือกในกลุ่มเดียวกัน
checkbox	เลือกได้หลายตัวเลือก
color	เลือกสี
submit	ส่งข้อมูลใน Form
reset	ล้างข้อมูลใน Form





ตัวอย่าง HTML Form



```
<body>
<div class="form-container">
  <h1>Student Register Form</h1>

  <form action="input_example.html" method="get">

    <!-- 1. text -->
    <label for="fullname">Full Name:</label>
    <input type="text" id="fullname" name="fullname" placeholder="Enter full name" required>

    <!-- 2. email -->
    <label for="email">Email:</label>
    <input type="email" id="email" name="email" placeholder="student@kku.ac.th" required>

    <!-- 3. password -->
    <label for="password">Password:</label>
    <input type="password" id="password" name="password" minlength="8" required>
```





ตัวอย่าง HTML Form



```
<!-- 4. date -->
```

```
<label for="birthday">Birthday:</label>
```

```
<input type="date" id="birthday" name="birthday">
```

```
<!-- 5. radio -->
```

```
<p><strong>Role:</strong></p>
```

```
<div class="option-group">
```

```
<input type="radio" id="user" name="role" value="user" checked>
```

```
<label for="user">User</label>
```

```
<input type="radio" id="admin" name="role" value="admin">
```

```
<label for="admin">Admin</label>
```

```
</div>
```

Student Register Form

Full Name:

Email:

Password:

Birthday:

Role:

☒ User ☐ Admin

Interested Topics:

☐ HTML ☐ CSS ☐ Web Security

Favorite Color:



Submit

Reset





ตัวอย่าง HTML Form



<!-- 6. checkbox -->

<p>Interested Topics:</p>

<div class="option-group">

<input type="checkbox" id="html" name="topic" value="html">

<label for="html">HTML</label>

<input type="checkbox" id="css" name="topic" value="css">

<label for="css">CSS</label>

<input type="checkbox" id="security" name="topic" value="security">

<label for="security">Web Security</label>

</div>

Student Register Form

Full Name:

Enter full name

Email:

student@kku.ac.th

Password:

Birthday:

mm/dd/yyyy

Role:

☒ User ☐ Admin

Interested Topics:

☐ HTML ☐ CSS ☐ Web Security

Favorite Color:



Submit

Reset





ตัวอย่าง HTML Form



```
<!-- 7. color -->
<label for="favcolor">Favorite Color:</label>
<input type="color" id="favcolor" name="favcolor" value="#008060">

<!-- Buttons -->
<div class="button-group">
  <input type="submit" value="Submit">
  <input type="reset" value="Reset">
</div>

</form>
</div>
</body>
```

Student Register Form

Full Name:

Email:

Password:

Birthday:

Role:

☒ User ☐ Admin

Interested Topics:

☐ HTML ☐ CSS ☐ Web Security

Favorite Color:



Submit

Reset





ตัวอย่าง HTML Form



GET Method Request

- ตัวอย่าง URL หลัง Submit

student_register.html?

fullname=Alice+Green&email=alice%40kku.ac.th&
password=123456&birthday=2026-06-29&
role=user&topic=html&favcolor=%23008060

ข้อสังเกต

- ข้อมูลแสดงใน Address Bar และสามารถคัดลอก/แก้ไขได้
- DevTools: Network → Query String Parameters
- ไม่เหมาะสมกับข้อมูลสำคัญ เช่น password หรือ role



Student Register Form

Full Name:

Email:

Password:

Birthday:

Role:
☒ User ☐ Admin

Interested Topics:
☐ HTML ☐ CSS ☐ Web Security

Favorite Color:





ตัวอย่าง HTML Form



Post Method Request

- ตัวอย่าง URL หลัง Submit
`student_register.html`
- Form Data ใน DevTools

```
fullname: Alice Green  
email: alice@kku.ac.th  
password: 123456  
birthday: 2026-06-29  
role: user  
topic: html  
favcolor: #008060
```



Student Register Form

Full Name:

Email:

Password:

Birthday:

Role:

☒ User ☐ Admin

Interested Topics:

☐ HTML ☐ CSS ☐ Web Security

Favorite Color:



Submit

Reset





ตัวอย่าง HTML Form



Post Method Request

ข้อสังเกต

- ข้อมูลไม่แสดงต่อท้าย URL
- DevTools: Network → Payload / Form Data
- POST ไม่ได้ปลอดภัยอัตโนมัติ ต้องใช้ HTTPS และตรวจสอบที่ Server

Student Register Form

Full Name:

Email:

Password:

Birthday:

Role:

☒ User ☐ Admin

Interested Topics:

☐ HTML ☐ CSS ☐ Web Security

Favorite Color:

Submit

Reset





อ้างอิง



- <https://www.w3schools.com/html/>
- <https://www.w3schools.com/css/>
- https://www.w3schools.com/html/html_forms.asp

